FlashInfer

Danyang Zhuo

Duke University

Some materials from Zihao's FlashInfer Slides

Logistics

- Programming assignment 4 due this Thursday, March 5
- Programming assignment 5 is posted
- Please get results for the course project and start writing the checkpoint 2
  - Please think about how to do live demo
  - Individual project: 10 minutes
  - Group project: 20 minutes

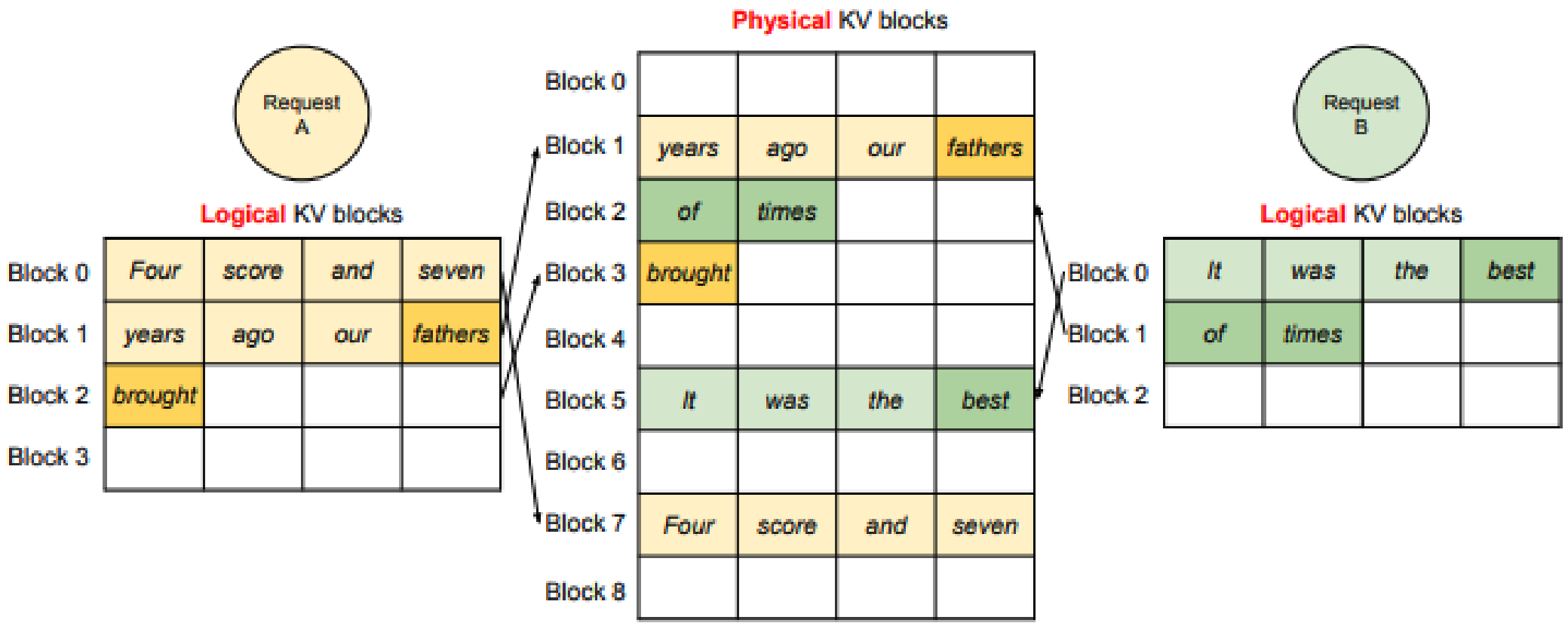
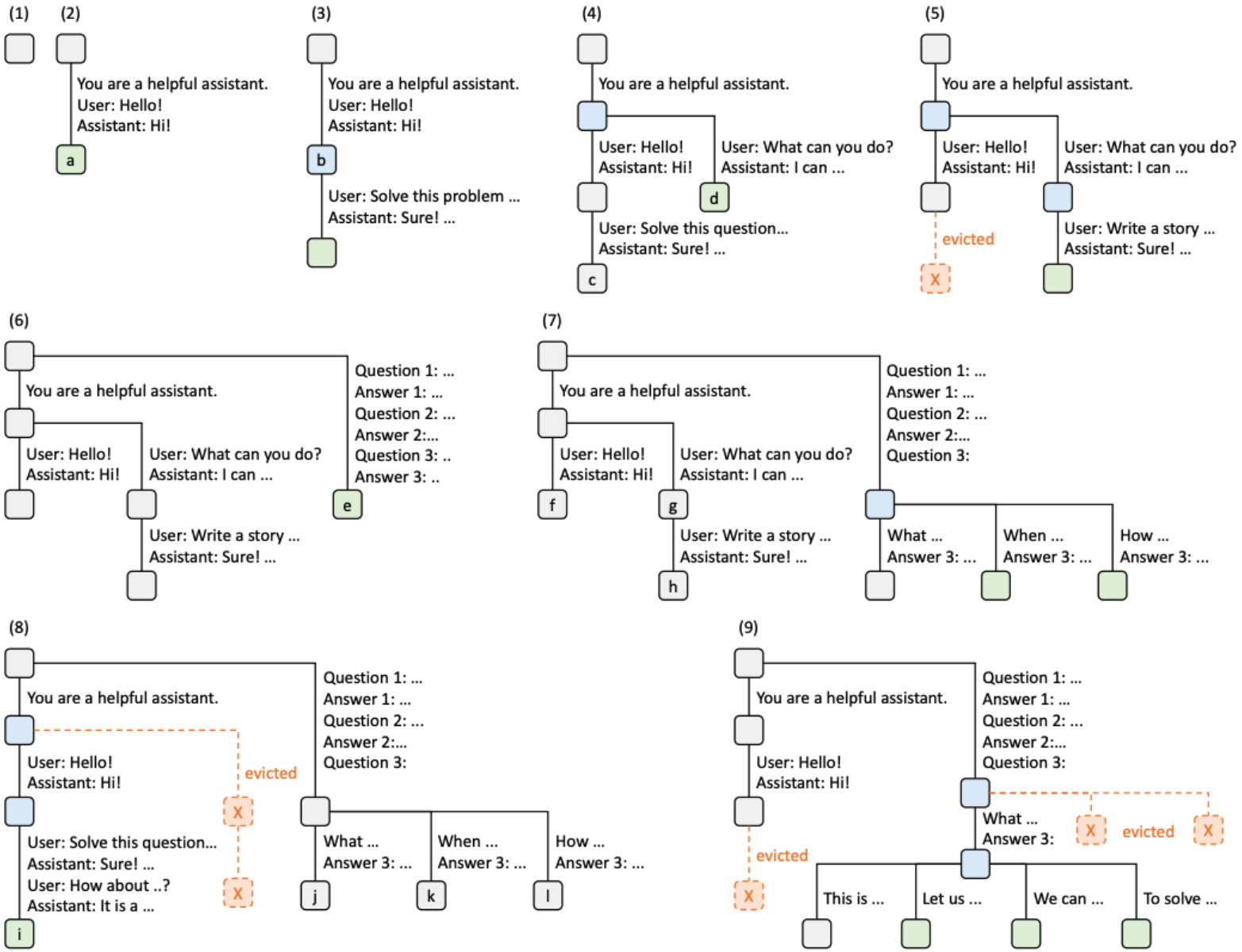


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

Paged Attention


(1)
(2)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
a
(3)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
b
User: Solve this problem ...
Assistant: Sure! ...
(4)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
User: What can you do?
Assistant: I can ...
d
User: Solve this question...
Assistant: Sure! ...
c
(5)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
User: What can you do?
Assistant: I can ...
evicted
X
User: Write a story ...
Assistant: Sure! ...
(6)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
User: What can you do?
Assistant: I can ...
User: Write a story ...
Assistant: Sure! ...
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3: ..
Answer 3: ..
e
(7)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
User: What can you do?
Assistant: I can ...
f
g
User: Write a story ...
Assistant: Sure! ...
h
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3:
What ...
Answer 3: ...
When ...
Answer 3: ...
How ...
Answer 3: ...
(8)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
evicted
X
User: Solve this question...
Assistant: Sure! ...
User: How about ..?
Assistant: It is a ...
X
i
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3:
What ...
Answer 3: ...
When ...
Answer 3: ...
How ...
Answer 3: ...
j
k
l
(9)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
evicted
X
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3:
What ...
Answer 3:
X
evicted
X
This is ...
Let us ...
We can ...
To solve ...

SGLang

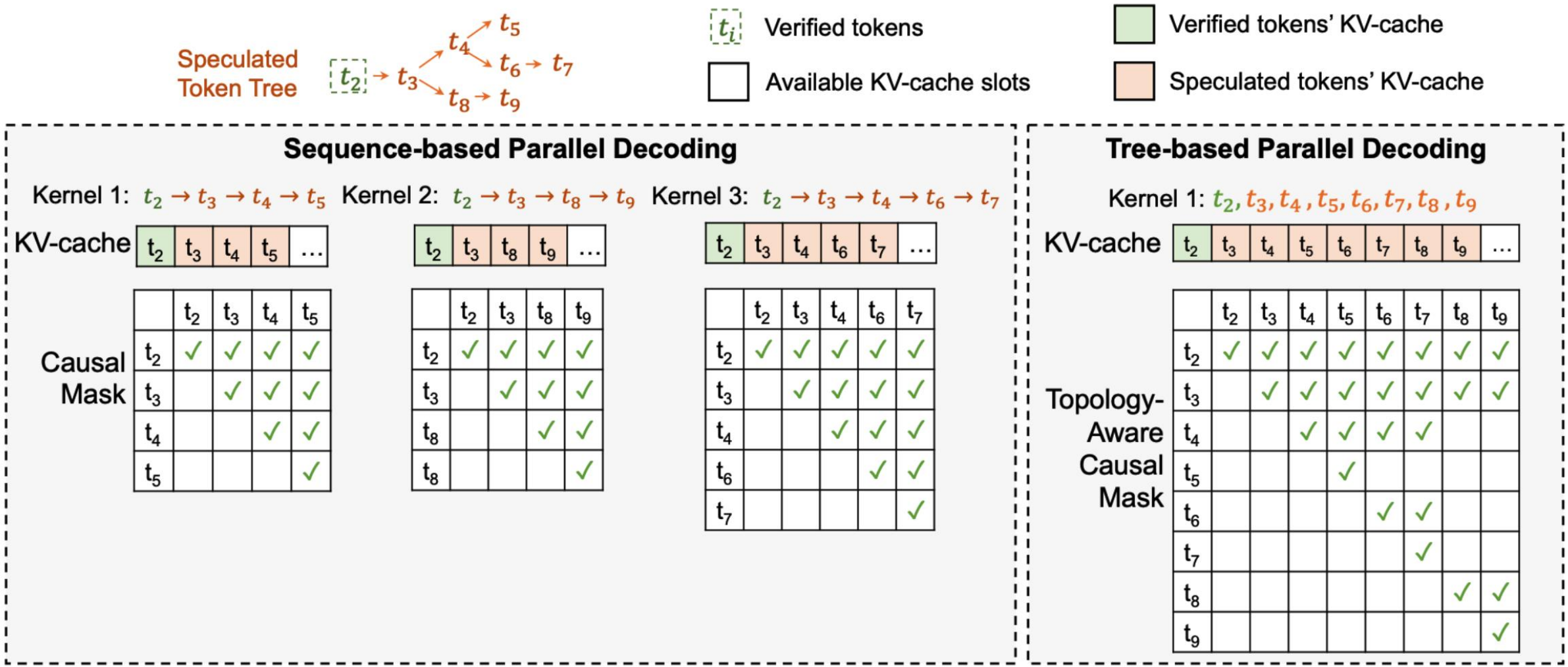
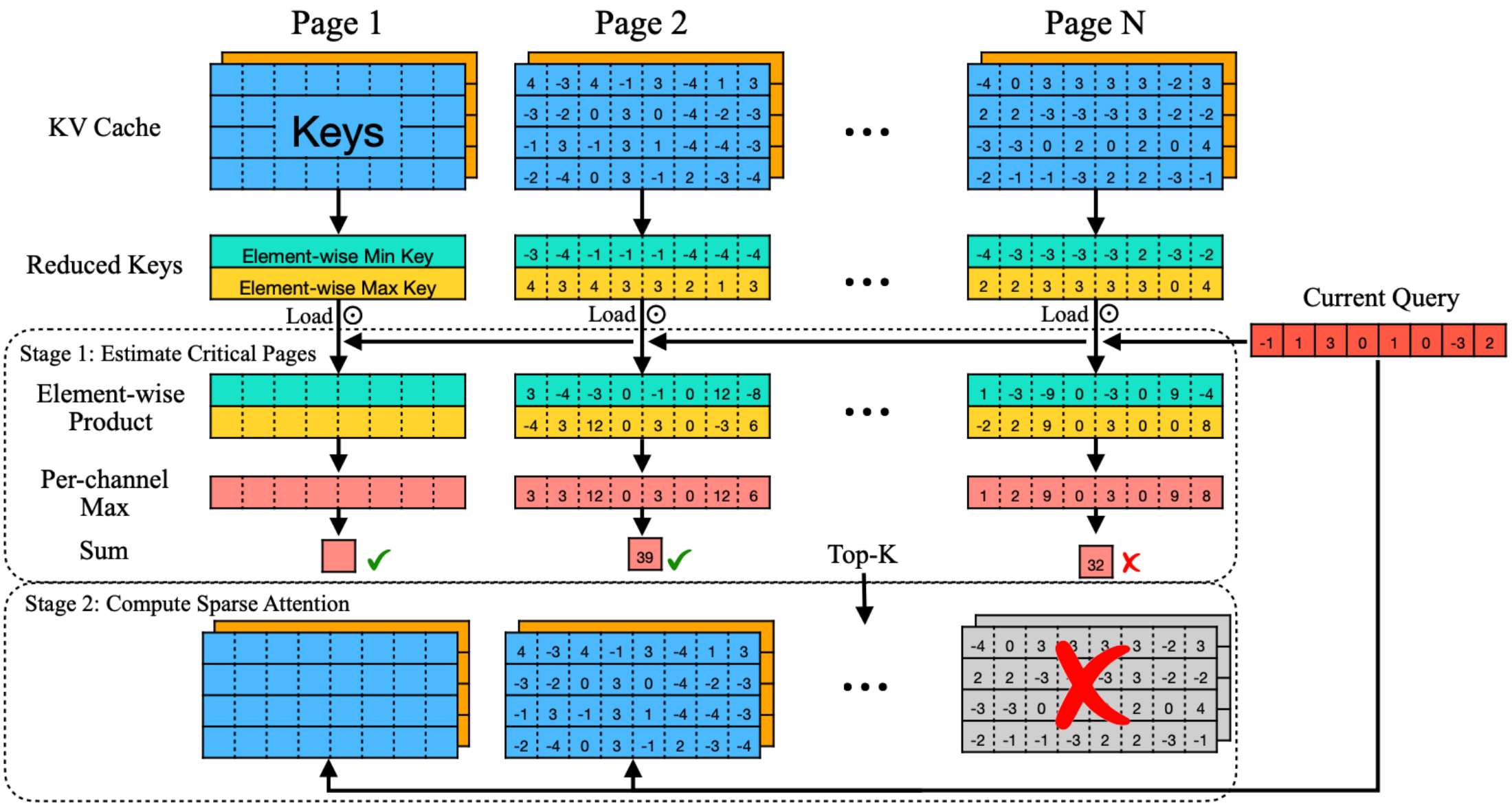


Figure 4. Comparing SpecInfer's tree-based parallel decoding with existing sequence-based decoding.


Page 1
Page 2
Page N
KV Cache
Keys
Reduced Keys
Element-wise Min Key
Element-wise Max Key
Load ⊙
Current Query
-1 1 3 0 1 0 -3 2
Stage 1: Estimate Critical Pages
Element-wise Product
Per-channel Max
Sum
39
32
Top-K
Stage 2: Compute Sparse Attention

Quest

Two major challenges in efficient attention

- Diverse workload patterns and input dynamics
  - Prefill
  - Batched decode
  - Speculative decoding

- Modern hardware implementation necessitate customization of attention operators
  - Storage format: paged attention, radix attention
  - Customized attentions: specialized masks, customized attentions score

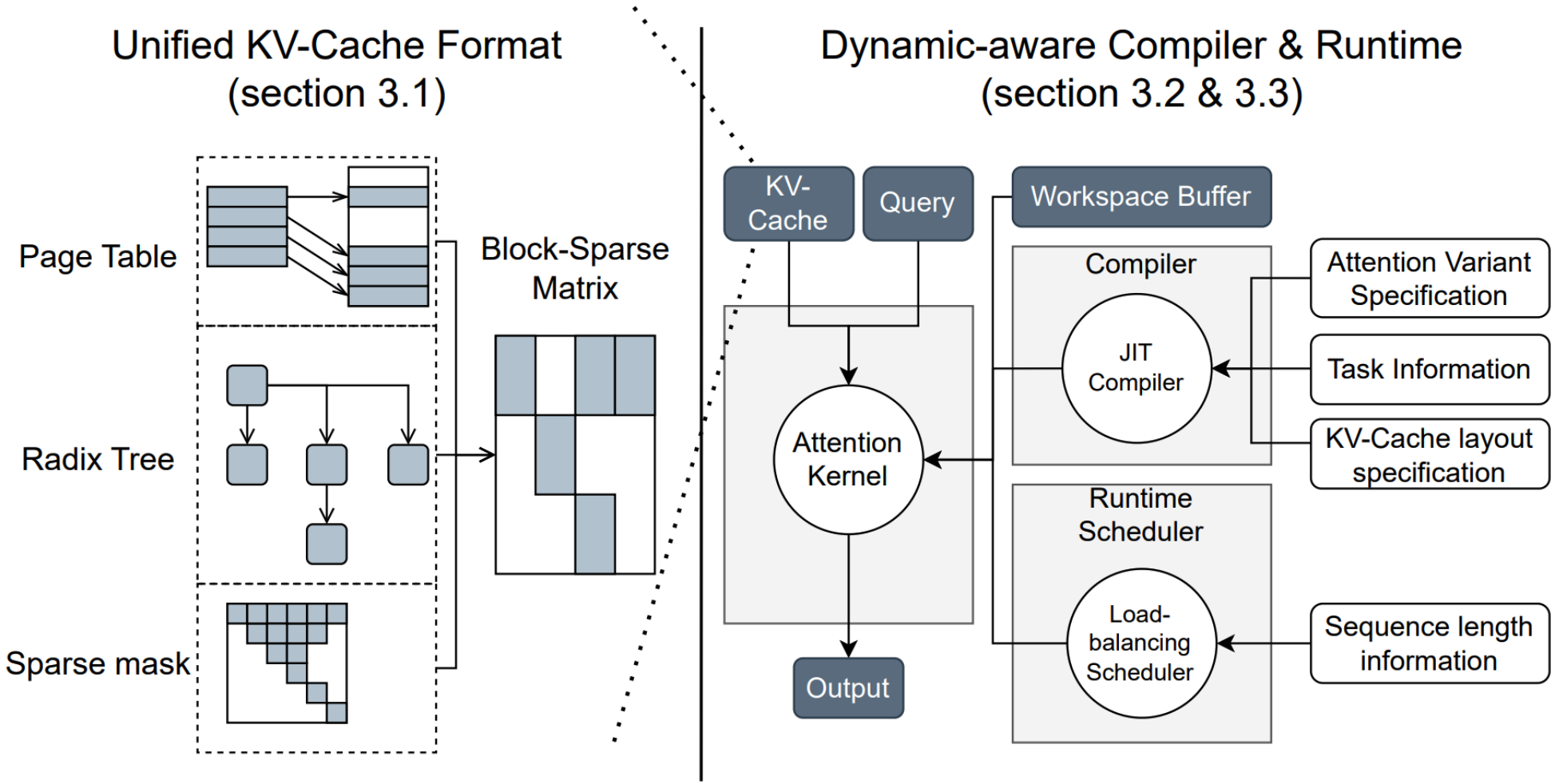


Figure 1. Overview of the FlashInfer system design: Attention variant specifications, task information and KV-cache layout specifics are provided at compile time for JIT compilation, while sequence length information is input at runtime for dynamic scheduling.

Block-Sparse Matrix as Unified Format

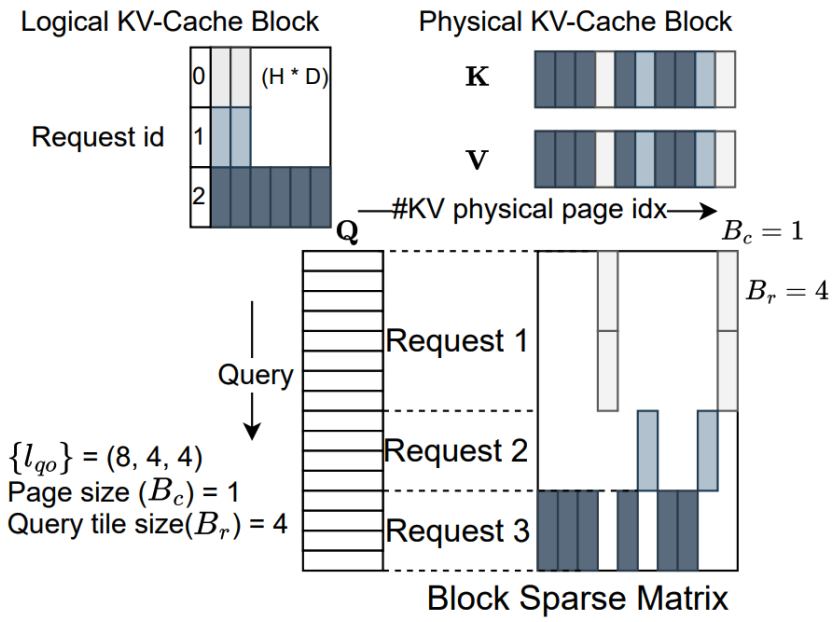


Figure 2. Representation of Page Table in BSR ($B_r = 4, B_c = 1$) format. The number of column blocks in the block sparse matrix corresponds to the total number of blocks allocated for the Page Table. Non-zero blocks represent KV-Cache pages accessed by queries.

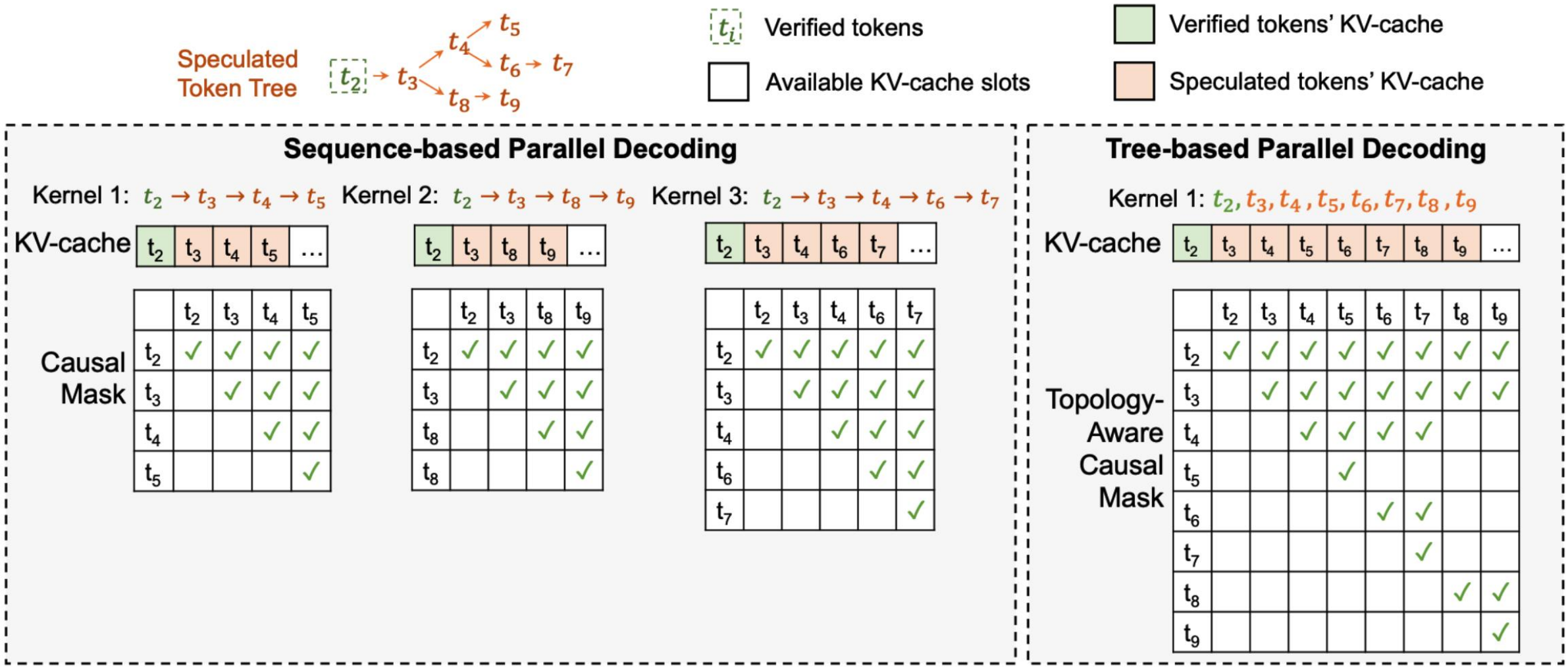
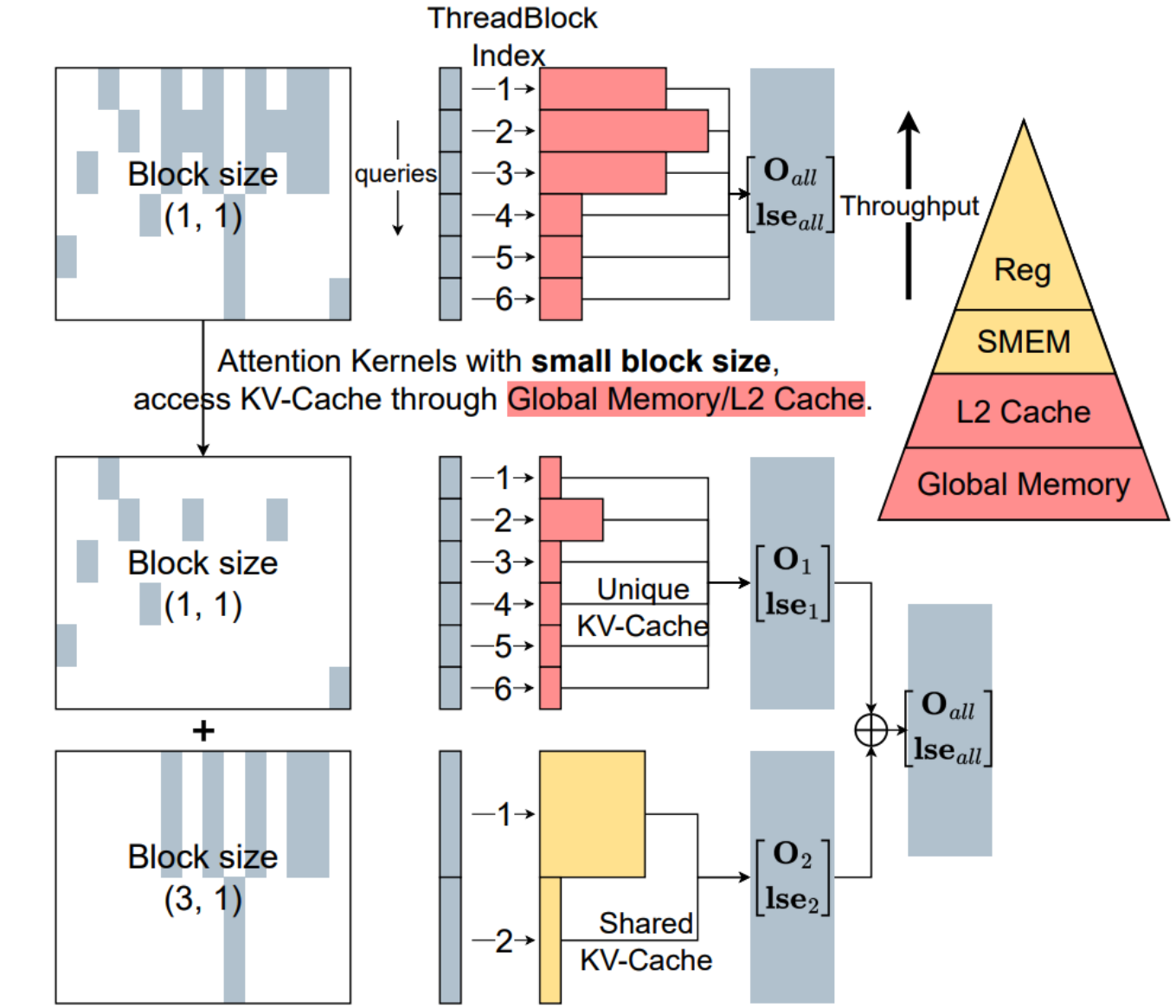


Figure 4. Comparing SpecInfer's tree-based parallel decoding with existing sequence-based decoding.



Composable Formats for Memory-Efficient Computation

Attention Kernels with **large block size**, requests inside the same block access common KV-Cache (e.g. shared-prefix) via Shared Memory/Registers

Compute Abstraction

- JIT Compiler
  - Customize attention variants by defining functions
  - Support continuous/sparse kv cache storage
  - Compile-time scheduling for different problem shapes

- Runtime
  - Variable-length dynamic scheduling
  - Programming interface compatible with open source LLM serving engines

Attention Specification in Python

```
spec_decl = r"""
template <typename Params_, typename KernelTraits_>
struct FlashSigmoid {
  using Params = typename Params_;
  using KernelTraits = typename KernelTraits_;
  static constexpr bool use_softmax = false;
  float scale, bias;
  FlashSigmoid(const Params& params, int batch_idx, uint8_t*
smem_ptr) {
    // Copy from CUDA constant memory to registers
    scale = params.scale;
    bias = params.bias;
  }
  ...
  float LogitsTransform(const Params& params, float
logit_score, int batch_idx, int qo_idx, int kv_idx, int
qo_head_idx, int kv_head_idx) {
    return 1. / (1. + expf(-(logits_score * scale + bias)));
  }
};
"""
attn_spec = AttentionSpec(
  "FlashSigmoid",
  dtype_q, dtype_kv, dtype_o, idtype, head_dim, is_sparse,
  additional_vars=[("scale", "float"), ("bias", "float")],
  additional_tensors=[],
  spec_decl=spec_decl
)
```

Part 1: Kernel Parameters Class

```
template <typename DTypeQ, typename DTypeKV, typename DTypeO,
typename IdType>
struct Params {
  DTypeQ* q;
  DTypeKV* k, v;
  DTypeO* o;
  float* lse;
  IdType* qo_indptr, kv_indptr, kv_indices, kv_seq_lens;
  ...
  // (generated) additional vars
  float scale;
  float bias;
};
```

Part 2: Kernel Traits class

```
struct KernelTraits {
  static constexpr HEAD_DIM = {{head_dim}};
  static constexpr IS_SPARSE = {{is_sparse}};
  ...
};
```

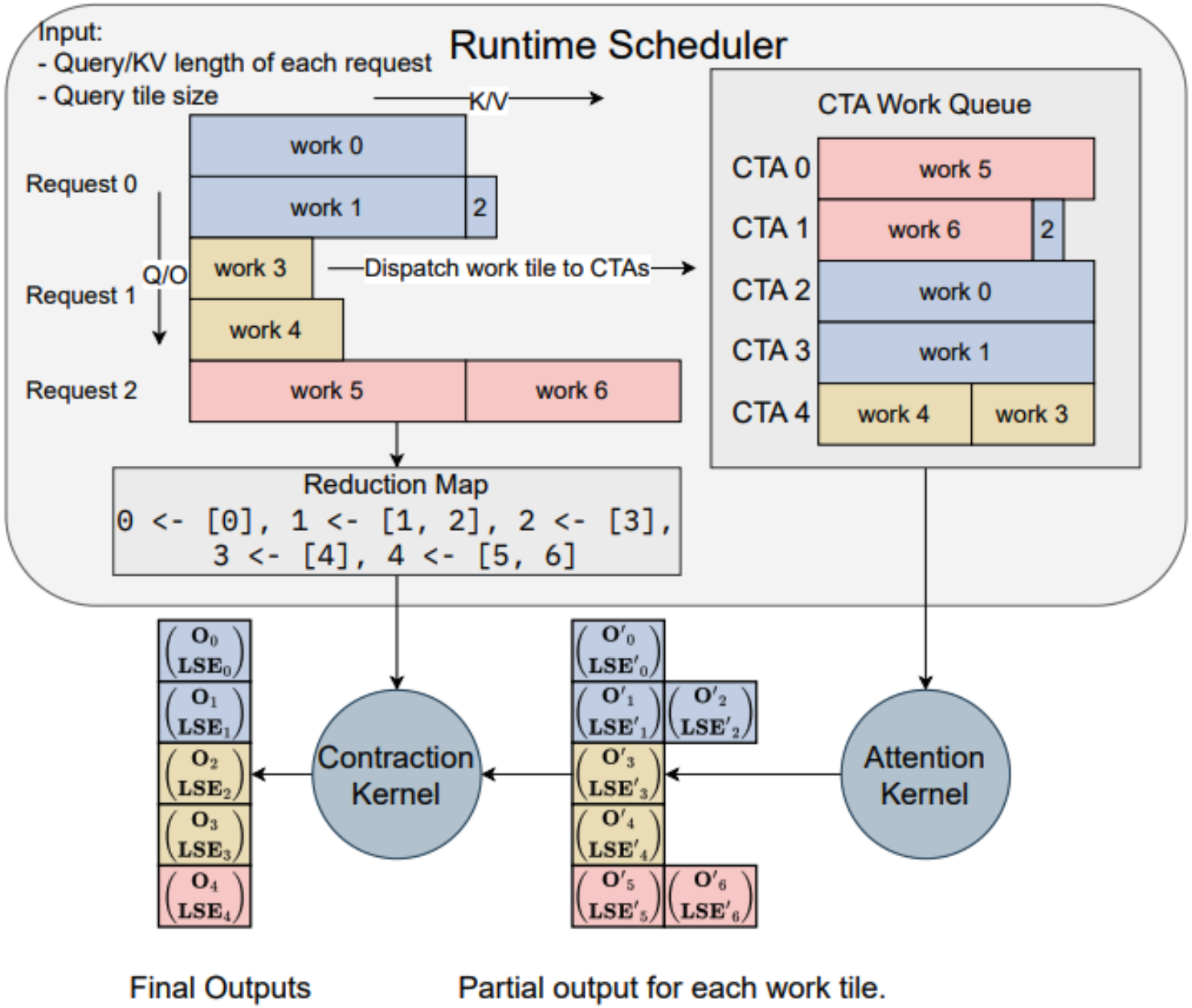
Part 3: Kernel Body

```
template <typename AttentionSpec>
__global__ KernelTemplate(
  AttentionSpec::Params params) {
  ...
  // Init attention specification class.
  AttentionSpec attn(params, batch_idx, smem_ptr);
  ...
  // Iterate over all elements inside the thread logits tile.
  for (int i = 0; i < size(logits_tile); ++i) {
    // convert register index i to qo_idx, kv_idx, etc.
    qo_idx = get<0>(logits_tile(i));
    kv_idx = get<1>(logits_tile(i));
    ...
    logits_tile(i) = attn.LogitsTransform(
      params, logits_tile(i),
      batch_idx, qo_idx, kv_idx,
      qo_head_idx, kv_head_idx);
  }
  ...
}
```

Part 4: Register custom operators in PyTorch

```
torch::Tensor attention_call(
  torch::Tensor q, torch::Tensor k, torch::Tensor v,
  ...
  float scale, float bias  // (generated) additional vars) {
  ...
  auto kernel = KernelTemplate<FlashSigmoid<Params<{{dtype_q}},
{{dtype_kv}}, {{dtype_o}}, {{idtype}}>, KernelTraits>>;
  ...
}
// Register torch custom ops
TORCH_LIBRARY_IMPL("FlashSigmoid", CUDA, m) {
  m.impl("run", &attention_call);
}
```

Figure 5. JIT compiler for attention variants in FlashInfer, featuring CUDA code strings defining variant functors, additional variables/tensors, and data types, used to populate kernel templates. Corresponding codes share highlighting.


Input:
- Query/KV length of each request
- Query tile size
Runtime Scheduler
K/V
Q/O
Request 0
Request 1
Request 2
work 0
work 1
2
work 3
work 4
work 5
work 6
Dispatch work tile to CTAs
CTA Work Queue
CTA 0
CTA 1
CTA 2
CTA 3
CTA 4
work 5
work 6
2
work 0
work 1
work 4
work 3
Reduction Map
0 <- [0], 1 <- [1, 2], 2 <- [3],
3 <- [4], 4 <- [5, 6]
Contraction Kernel
Attention Kernel
Final Outputs
Partial output for each work tile.

Algorithm 1 FlashInfer's balanced scheduling algorithm

Input: $\{l_{qo}(i), l_{kv}(i)\}_i$, query tile size T_q.

Define the cost of a tile l_q, l_{kv} as (α, β are hyperparameters):

$$\text{cost}(l_q, l_{kv}) = \alpha l_q + \beta l_{kv}$$

Compute the maximum KV chunk size L_{kv} by

$$L_{kv} \leftarrow \frac{\sum_i \lceil \frac{l_{qo}(i)}{T_q} \rceil \cdot l_{kv}(i)}{\#\text{CTA}}$$

Split each query tile's KV into chunks, with maximum size L_{kv}, we assign each chunk a work index w, and the length of the chunk is $l_{kv}(w)$.

Let $W = \{(w, l_{kv}(w))\}$ and sort the entries in descending order of length.

$Q \leftarrow$ PriorityQueue($\{(c, 0)\}$) where c is the CTA index.

while $W \neq \emptyset$ **do**

  c, current_cost $\leftarrow Q.\text{pop}_{min}()$

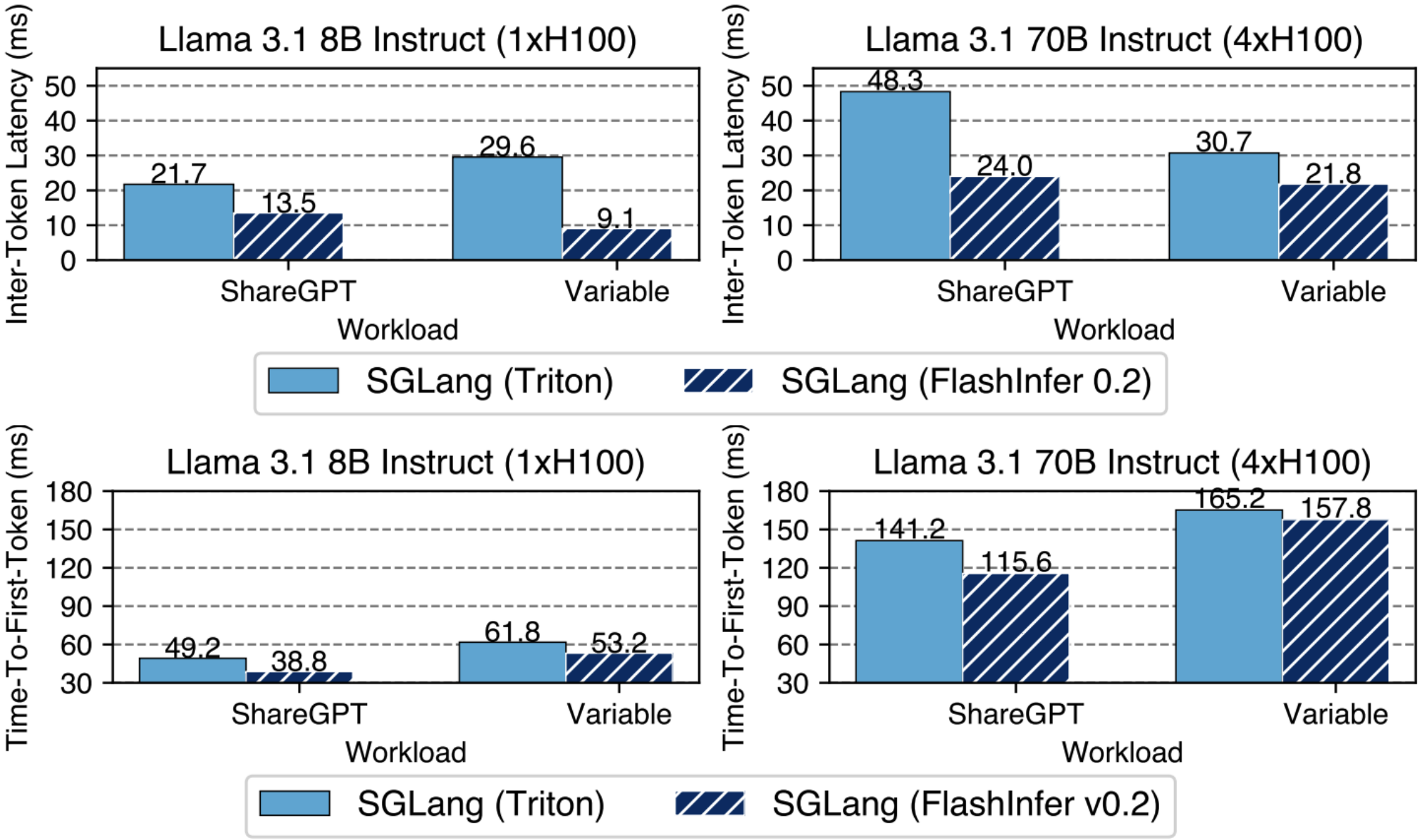
  $w, l_{kv}(w) \leftarrow W.\text{pop}()$

  new_cost $\leftarrow$ current_cost $+$ cost$(T_q, l_{kv}(w))$

  Assign chunk w to CTA c

  Q.push($(c$, new_cost$)$)

end while


Llama 3.1 8B Instruct (1xH100)
Inter-Token Latency (ms)
50
40
30
20
10
0
21.7
13.5
29.6
9.1
ShareGPT
Variable
Workload
Llama 3.1 70B Instruct (4xH100)
Inter-Token Latency (ms)
48.3
24.0
30.7
21.8
ShareGPT
Variable
Workload
SGLang (Triton)
SGLang (FlashInfer 0.2)
Llama 3.1 8B Instruct (1xH100)
Time-To-First-Token (ms)
180
150
120
90
60
30
49.2
38.8
61.8
53.2
ShareGPT
Variable
Workload
Llama 3.1 70B Instruct (4xH100)
Time-To-First-Token (ms)
141.2
115.6
165.2
157.8
ShareGPT
Variable
Workload
SGLang (Triton)
SGLang (FlashInfer v0.2)

DeepSeek-V3 Technical Report

DeepSeek-AI

research@deepseek.com

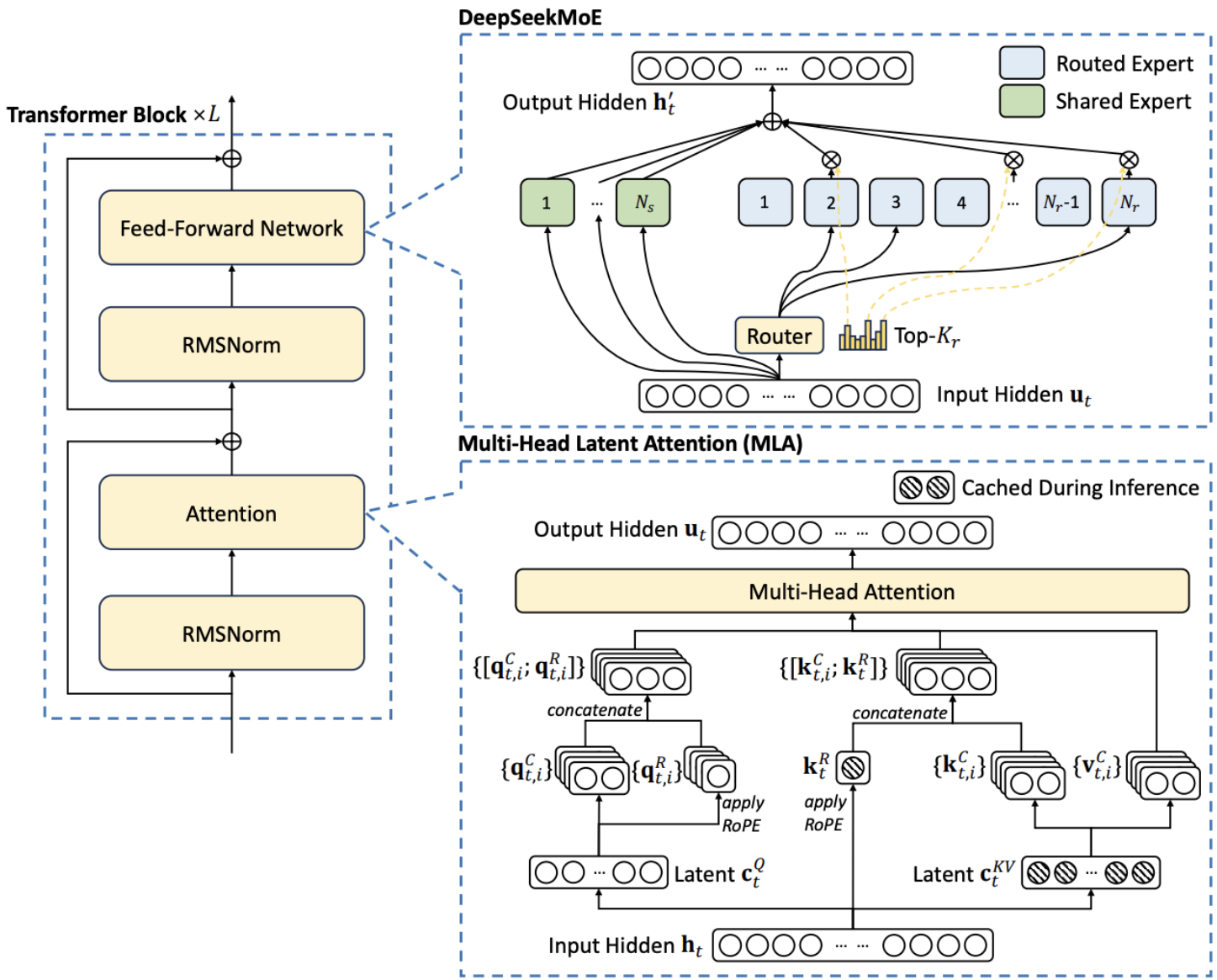


Figure 2 | Illustration of the basic architecture of DeepSeek-V3. Following DeepSeek-V2, we adopt MLA and DeepSeekMoE for efficient inference and economical training.

Training

- First stage
  - Max context window is 32K
- Second stage
  - Max context window is 128K
- Supervised Fine Tuning + Reinforcement Learning

Training Costs	Pre-Training	Context Extension	Post-Training	Total
in H800 GPU Hours	2664K	119K	5K	2788K
in USD	$5.328M	$0.238M	$0.01M	$5.576M

Table 1 | Training costs of DeepSeek-V3, assuming the rental price of H800 is $2 per GPU hour.

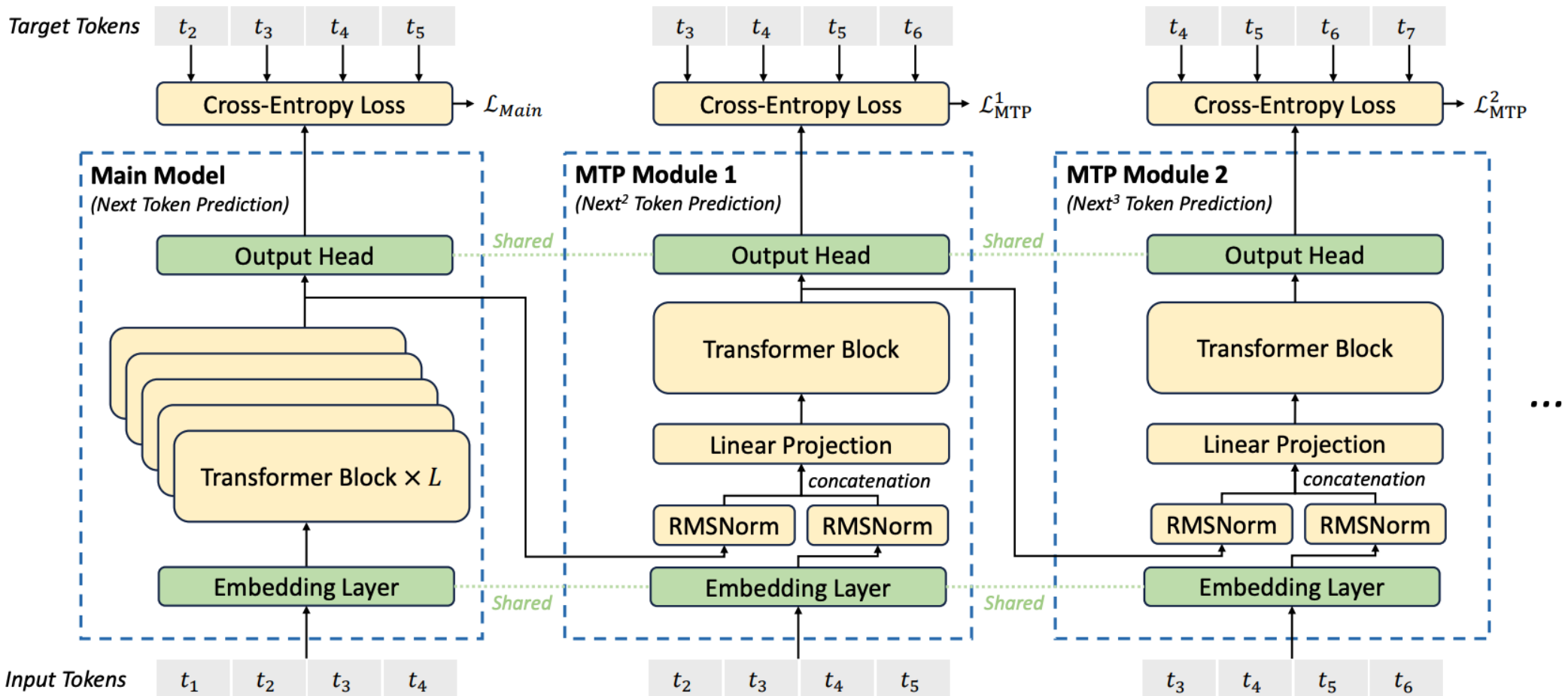


Figure 3 | Illustration of our Multi-Token Prediction (MTP) implementation. We keep the complete causal chain for the prediction of each token at each depth.

FP8 Mixed Precision Training

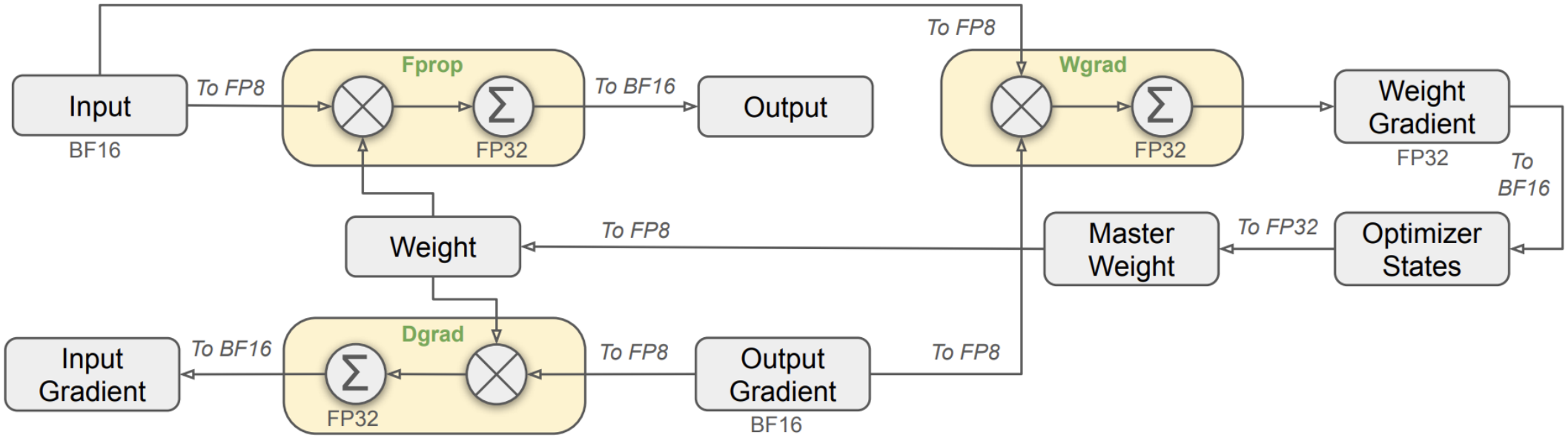


Figure 6 | The overall mixed precision framework with FP8 data format. For clarification, only the `Linear` operator is illustrated.

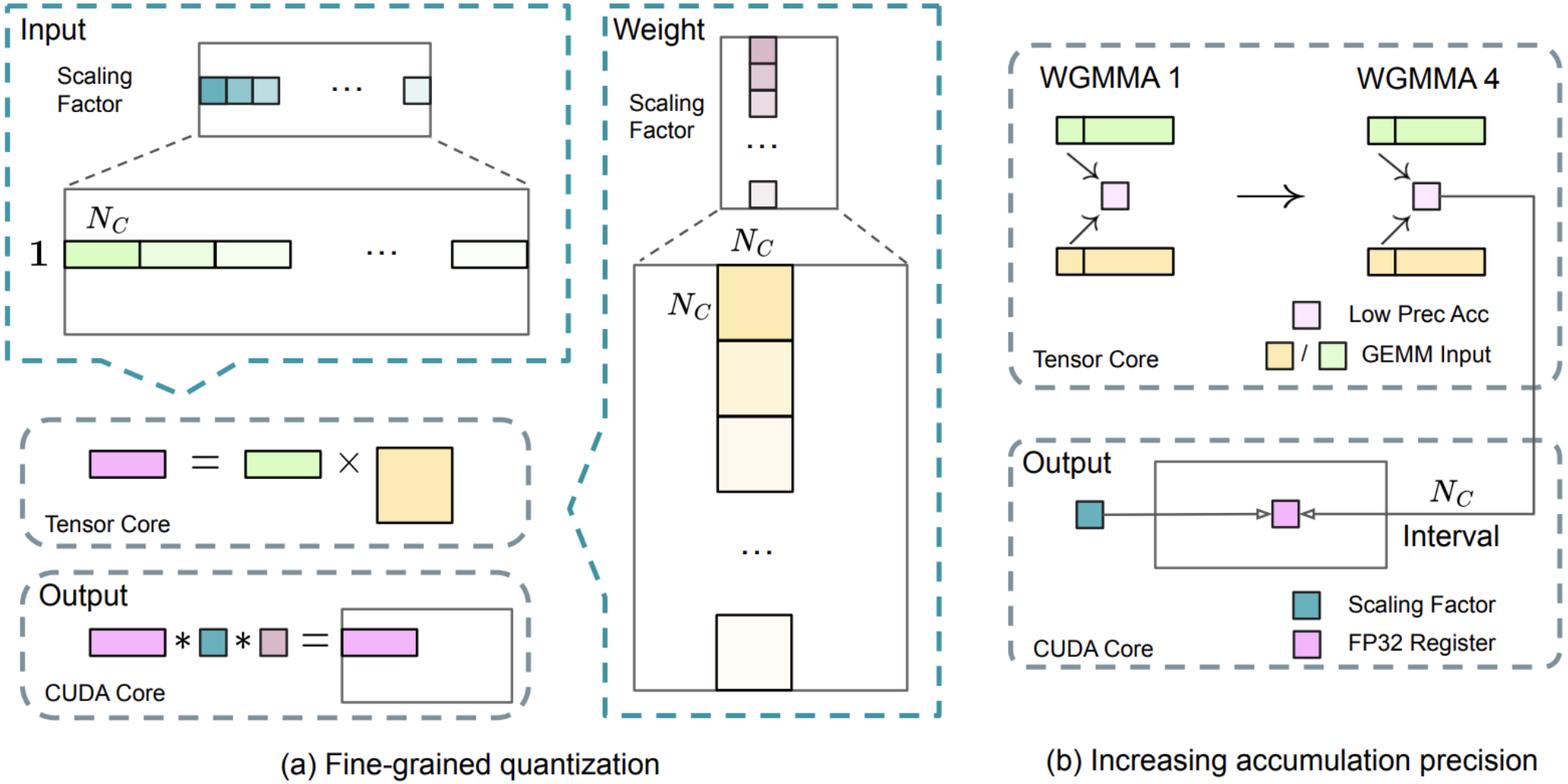
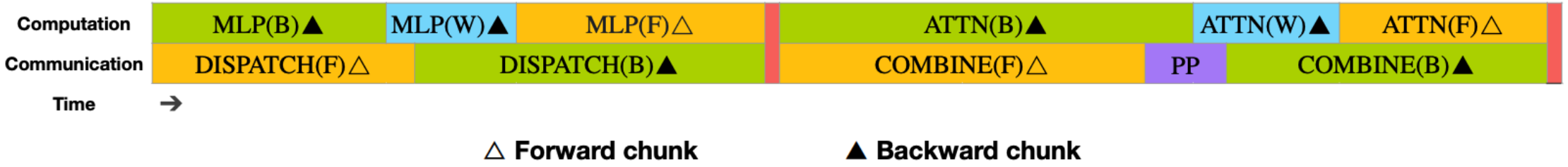


Figure 7 | (a) We propose a fine-grained quantization method to mitigate quantization errors caused by feature outliers; for illustration simplicity, only `Fprop` is illustrated. (b) In conjunction with our quantization strategy, we improve the FP8 GEMM precision by promoting to CUDA Cores at an interval of $N_C = 128$ elements MMA for the high-precision accumulation.

Device 0
Device 1
Device 2
Device 3
Device 4
Device 5
Device 6
Device 7
Time
Forward
Backward
Backward for input
Backward for weights
Overlapped forward & Backward


Computation
MLP(B)▲
MLP(W)▲
MLP(F)△
ATTN(B)▲
ATTN(W)▲
ATTN(F)△
Communication
DISPATCH(F)△
DISPATCH(B)▲
COMBINE(F)△
PP
COMBINE(B)▲
Time
△ Forward chunk
▲ Backward chunk